

Java Strings

Last Updated : 12 Dec, 2024



In Java, **String** is the type of objects that can store the sequence of characters enclosed by double quotes and every character is stored in 16 bits i.e using UTF 16-bit encoding. A string acts the same as an array of characters. Java provides a robust and flexible API for handling strings, allowing for various operations such as **concatenation**, **comparison**, and **manipulation**.

Example:

```
String name = "Geeks";
```

```
String num = "1234"
```

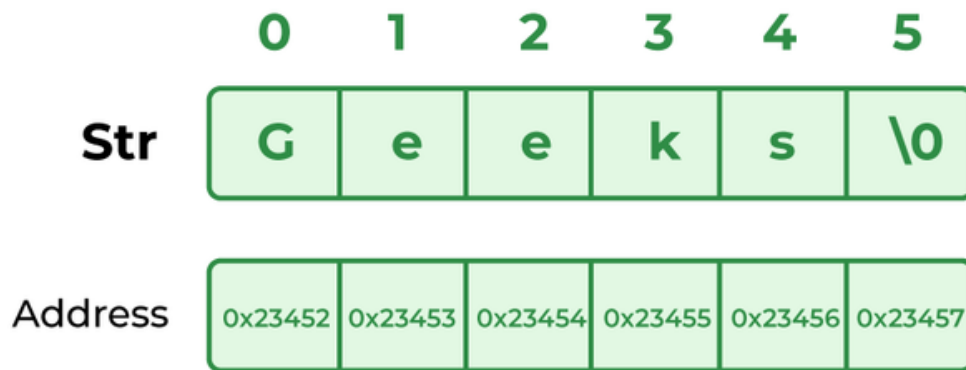
Java



```
1 // Java Program to demonstrate String
2 public class Geeks {
3
4     // Main Function
5     public static void main(String args[])
6     {
7
8         // creating Java string using new keyword
9         String str = new String("GeeksforGeeks");
10
11         System.out.println(str);
12     }
13 }
```

Output

```
GeeksforGeeks
```



String Example in Java

Ways of Creating a Java String

There are two ways to create a string in Java:

- String Literal
- Using new Keyword

Syntax:

```
<String_Type> <string_variable> = "<sequence_of_string>";
```

1. String literal (Static Memory)

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

Example:

```
String demoString = "GeeksforGeeks";
```

2. Using new keyword (Heap Memory)

- `String s = new String("Welcome");`
- In such a case, JVM will create a new string object in normal (non-pool) heap memory and the literal "Welcome" will be placed in the string constant pool. The variable `s` will refer to the object in the heap (non-pool)

In the given example only one object will be created. Firstly JVM will not find any string object with the value "Welcome" in the string constant pool, so it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.

Example:

```
String demoString = new String ("GeeksforGeeks");
```

Interfaces and Classes in Strings in Java

- [CharBuffer](#): This class implements the `CharSequence` interface. This class is used to allow character buffers to be used in place of `CharSequences`. An example of such usage is the regular-expression package `java.util.regex`.
- [String](#): It is a sequence of characters. In Java, objects of `String` are immutable which means a constant and cannot be changed once created.

CharSequence Interface

`CharSequence` Interface is used for representing the sequence of Characters in Java. Classes that are implemented using the `CharSequence` interface are mentioned below and these provides much of functionality like substring, lastoccurence, first occurrence, concatenate , toupper, tolower etc.

1. `String`
2. `StringBuffer`
3. `StringBuilder`

1. String

String is an immutable class which means a constant and cannot be changed once created and if wish to change , we need to create an new object and even the functionality it provides like toupper, tolower, etc all these return a new object , its not modify the original object. It is automatically thread safe.

Syntax

```
// Method 1
String str= "geeks";

// Method 2
String str= new String("geeks")
```

2. StringBuffer

[StringBuffer](#) is a peer class of **String**, it is mutable in nature and it is thread safe class , we can use it when we have multi threaded environment and shared object of string buffer i.e, used by mutiple thread. As it is thread safe so there is extra overhead, so it is mainly used for multithreaded program.

Syntax:

```
StringBuffer demoString = new StringBuffer("GeeksforGeeks");
```

3. StringBuilder

[StringBuilder](#) in Java represents an alternative to String and StringBuffer Class, as it creates a mutable sequence of characters and it is not thread safe. It is used only within the thread , so there is no extra overhead , so it is mainly used for single threaded program.

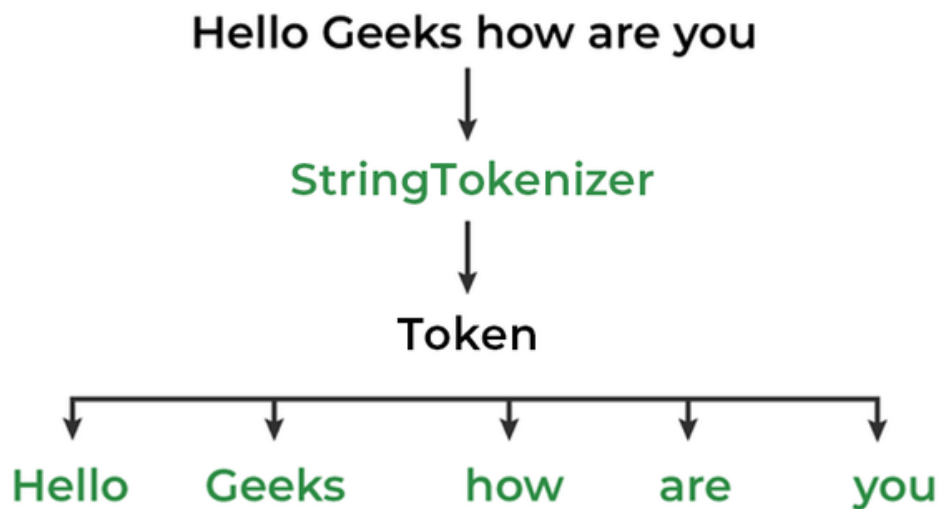
Syntax:

```
StringBuilder demoString = new StringBuilder(); demoString.append("GFG");
```

4. StringTokenizer

[StringTokenizer](#) class in Java is used to break a string into tokens.

Example:



A StringTokenizer object internally maintains a current position within the string to be tokenized. Some operations advance this current position past the characters processed. A token is returned by taking a substring of the string that was used to create the StringTokenizer object.

StringJoiner is a class in [java.util](#) package which is used to construct a sequence of characters(strings) separated by a delimiter and optionally starting with a supplied prefix and ending with a supplied suffix. Though this can also be done with the help of the StringBuilder class to append a delimiter after each string, StringJoiner provides an easy way to do that without much code to write.

Syntax:

```
public StringJoiner(CharSequence delimiter)
```

Above we saw we can create a string by String Literal.

```
String demoString = "Welcome";
```

Here the JVM checks the String Constant Pool. If the string does not exist, then a new string instance is created and placed in a pool. If the string exists, then it will

not create a new object. Rather, it will return the reference to the same instance. The cache that stores these string instances is known as the String Constant pool or String Pool. In earlier versions of Java up to JDK 6 String pool was located inside PermGen(Permanent Generation) space. But in JDK 7 it is moved to the main heap area.

Immutable String in Java

In Java, string objects are immutable. Immutable simply means unmodifiable or unchangeable. Once a string object is created its data or state can't be changed but a new string object is created.

Below is the implementation of the topic:

Java

```
1 // Java Program to demonstrate Immutable String in Java
2 import java.io.*;
3
4 class GFG {
5     public static void main(String[] args)
6     {
7         String s = "Sachin";
8
9         // concat() method appends
10        // the string at the end
11        s.concat(" Tendulkar");
12
13        // This will print Sachin
14        // because strings are
15        // immutable objects
16        System.out.println(s);
17    }
18 }
```

Output

Sachin

Here Sachin is not changed but a new object is created with “Sachin Tendulkar”. That is why a string is known as immutable.

As you can see in the given figure that two objects are created but s reference variable still refers to “Sachin” and not to “Sachin Tendulkar”. But if we explicitly assign it to the reference variable, it will refer to the “Sachin Tendulkar” object.

Example:

Java

```
1 // Java Program to demonstrate Explicitly assigned
  strings
2 import java.io.*;
3
4 class GFG {
5     public static void main(String[] args)
6     {
7         String name = "Sachin";
8         name = name.concat(" Tendulkar");
9         System.out.println(name);
10    }
11 }
```

Output

Sachin Tendulkar

Memory Allotment of Java String

Whenever a String Object is created as a literal, the object will be created in the String constant pool. This allows JVM to optimize the initialization of String literal.

Example:

```
String demoString = "Geeks";
```

The string can also be declared using a **new** operator i.e. dynamically allocated. In case of String are dynamically allocated they are assigned a new memory location in the heap. This string will not be added to the String constant pool.

Example:

```
String demoString = new String("Geeks");
```

If you want to store this string in the constant pool then you will need to “intern” it.

Example:

```
// this will add the string to string constant pool.  
String internedString = demoString.intern();
```

It is preferred to use String literals as it allows JVM to optimize memory allocation.

An example that shows how to declare a String

Java

```
1 // Java code to illustrate String  
2 import java.io.*;  
3 import java.lang.*;  
4  
5 class Test {  
6     public static void main(String[] args) {  
7  
8         // Declare String without using new operator  
9         String name = "GeeksforGeeks";  
10  
11        // Prints the String.  
12        System.out.println("String name = " + name);  
13  
14        // Declare String using new operator  
15        String newString = new String("GeeksforGeeks");  
16  
17        // Prints the String.
```



```
18         System.out.println("String newString = " +
19         newString);
20     }
```

Output

```
String name = GeeksforGeeks
String newString = GeeksforGeeks
```

Note: String Object is created in Heap area and Literals are stored in special memory area known as string constant pool.

Why did the String pool move from PermGen to the normal heap area?

PermGen space is limited, the default size is just 64 MB. It was a problem with creating and storing too many string objects in PermGen space. That's why the String pool was moved to a larger heap area. To make Java more memory efficient, the concept of string literal is used. By the use of the 'new' keyword, The JVM will create a new string object in the normal heap area even if the same string object is present in the string pool.

For example:

```
String demoString = new String("Bhubaneswar");
```

Let us have a look at the concept with a Java program and visualize the actual JVM memory structure:

Below is the implementation of the above approach:

Java



1

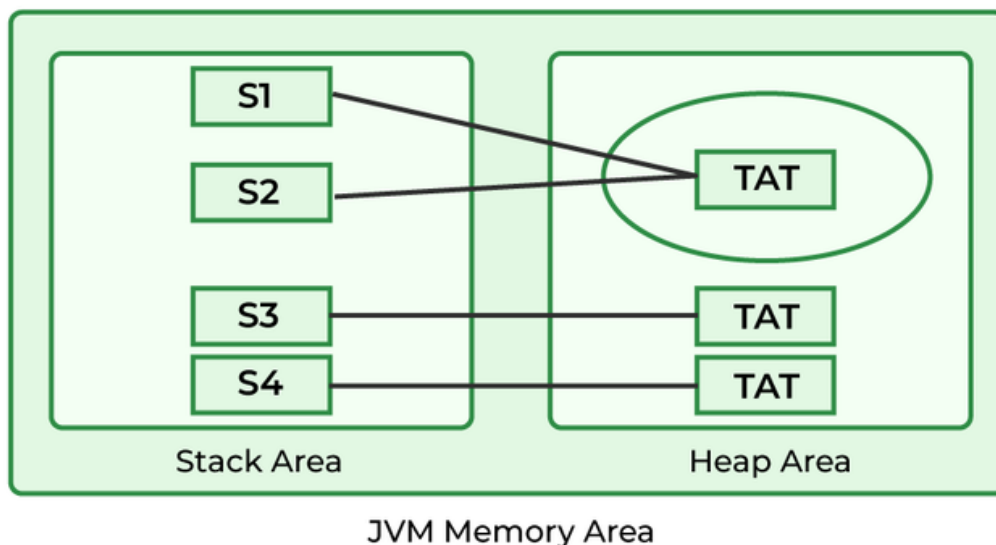
```
// Java program and visualize the
```



```
2 // actual JVM memory structure
3 // mentioned in image
4 class StringStorage {
5     public static void main(String args[]){
6
7         // Declaring Strings
8         String s1 = "TAT";
9         String s2 = "TAT";
10        String s3 = new String("TAT");
11        String s4 = new String("TAT");
12
13        // Printing all the Strings
14        System.out.println(s1);
15        System.out.println(s2);
16        System.out.println(s3);
17        System.out.println(s4);
18    }
19 }
```

Output

TAT
TAT
TAT
TAT



String Pool in Java

Note: All objects in Java are stored in a heap. The reference variable is to the object stored in the stack area or they can be contained in other objects which puts them in the heap area also.

Example 1:

Java

```
1 // Construct String from subset of char array
2
3 // Driver Class
4 class GFG {
5     public static void main(String args[]) {
6
7         // Creating Byte ASCII Array
8         byte ascii[] = { 71, 70, 71 };
9
10        // Creating String using byte array
11        String firstString = new String(ascii);
12        System.out.println(firstString);
13
14        // Creating String using byte array with
15        // Start index to End Index
16        String secondString = new String(ascii, 1, 2);
17        System.out.println(secondString);
18    }
19 }
```

Output

```
GFG
FG
```

Example 2:

Java



```
1 // Construct one string from another
2
3 class GFG {
4     public static void main(String args[]) {
5
6         // Character Array
7         char characters[] = { 'G', 'f', 'g' };
8
9         // Creating new String using Character Array
10        String firstString = new String(characters);
11
12        // Creating new String using another String
13        String secondString = new String(firstString);
14
15        System.out.println(firstString);
16        System.out.println(secondString);
17    }
18 }
```

Output

```
Gfg
Gfg
```

FAQs – Java String

What are strings in Java?

Strings are the types of objects which can store characters as elements.

Why string objects are immutable in Java?

Because java uses the concept of string literal. Suppose there are 5 reference variables, all refer to one object "Sachin". If one reference variable changes the value of the object, it will be affected by all the reference variables. That is why string objects are immutable in Java.

Why Java uses the concept of string literal?

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

String in Java

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Next Article >](#)[Why Java Strings are Immutable?](#)

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

🕒 4 min read

Java Strings

Java Strings

In Java, String is the type of objects that can store the sequence of characters enclosed by double quotes and every character is stored in 16 bits i.e using UTF 16-bit encoding. A string acts the same as an arra...

🕒 9 min read

Why Java Strings are Immutable?

In Java, strings are immutable means their values cannot be changed once they are created. This feature enhances performance, security, and thread safety. In this article, we will explain why strings are...

🕒 3 min read

Java String concat() Method with Examples

The string concat() method concatenates (appends) a string to the end of another string. It returns the combined string. It is used for string concatenation in Java. It returns NullPointerException if any one of...

🕒 4 min read

String class in Java

String is a sequence of characters. In Java, objects of the String class are immutable which means they cannot be changed once created. In this article, we will learn about the String class in Java. Example of...

🕒 7 min read

StringBuffer class in Java

StringBuffer is a class in Java that represents a mutable sequence of characters. It provides an alternative to the immutable String class, allowing you to modify the contents of a string without...

🕒 12 min read

StringBuilder Class in Java with Examples

StringBuilder in Java represents a mutable sequence of characters. Since the String Class in Java creates an immutable sequence of characters, the StringBuilder class provides an alternative to String Class, a...

🕒 6 min read

String vs StringBuilder vs StringBuffer in Java

A string is a sequence of characters. In Java, objects of String are immutable which means a constant and cannot be changed once created. Initializing a String is one of the important pillars required as a...